Exception Handling

Exception Handling in Java is a mechanism used to handle runtime errors so that the normal flow of the program can continue.

An **Exception** is an unwanted event that occurs during program execution, such as dividing by zero, accessing an invalid array index, or opening a non-existing file.

Why Exception Handling?

- Prevents program termination due to errors.
- Maintains normal program flow.
- Improves program reliability and readability.

Without Exception Handling

```
public class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        System.out.println(a / b);
        System.out.println("Program Continues...");
    }
}
```

Compiler gives no error.

Program crashes at runtime.

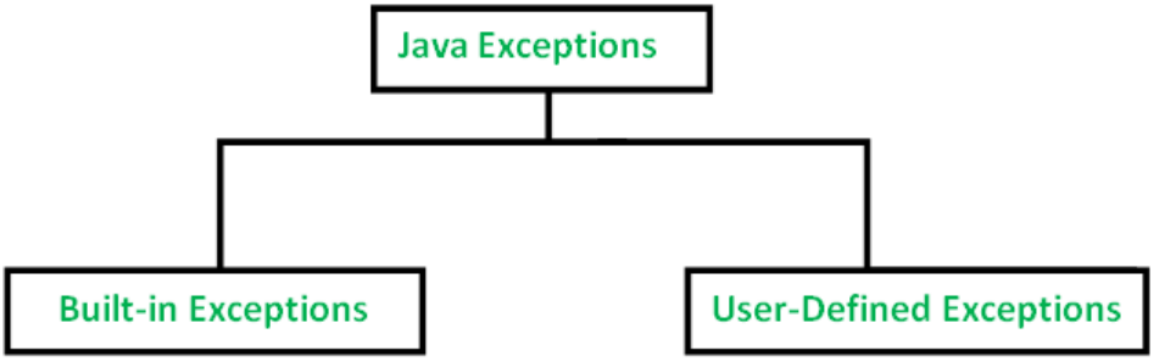
"Program Continues..." is not printed.

With Exception Handling

```
public class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        try {
            System.out.println(a / b);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }
        System.out.println("Program Continues...");
    }
}
```

Cannot divide by zero

Program Continues...



Types of Built-in Exceptions

Checked Exceptions

A **checked exception** is an exception that is **checked by the Java compiler**

```
//Without Checked Exception
import java.io.*;
public class Main {
    public static void main(String[] args) {
        FileReader f = new FileReader("abc.txt");
    }
}
//Compilation Error
```

```
// With Checked Exception: FileNotFoundException
import java.io.*;
class Main {
    public static void main(String[] args) {
        try {
            FileReader file = new FileReader("data.txt");
        } catch (FileNotFoundException e) {
            System.out.println("File not found");
        }
    }
}
File not found
```

```
// ClassNotFoundException
public class Main {
    public static void main(String[] args) {
        try {
            Class.forName("Employee");
            System.out.println("Class Found");
        } catch (ClassNotFoundException e) {
            System.out.println("Class Not Found");
        }
    }
```

```
}
```

Class Not Found

Unchecked Exceptions (Checked at runtime)

```
// ArithmeticException (Divide by Zero)
class UncheckedExceptionDemo {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        try {
            int result = a / b;
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        }
    }
}
```

```
// NullPointerException
public class Demo {
    public static void main(String[] args) {
        String name = null;
        System.out.println(name.length());
    }
}
```

```
// ArrayIndexOutOfBoundsException
public class Demo {
    public static void main(String[] args) {
        int a[] = {10, 20, 30};
        System.out.println(a[5]);
    }
}
```

```
// StringIndexOutOfBoundsException
public class Demo {
    public static void main(String[] args) {
        String s = "Java";
        System.out.println(s.charAt(10));
    }
}
```

NumberFormatException

```
//Occurs when a string cannot be converted to a number
public class Demo {
    public static void main(String[] args) {
        String s = "ABC";
        int n = Integer.parseInt(s);
```

```
        System.out.println(n);
    }
}
```

IllegalArgumentException

```
//Occurs when a method receives an illegal argument
public class Demo {
    public static void main(String[] args) {
        Thread t = new Thread();
        t.setPriority(15);   // Valid priorities: 1 to 10
    }
}
```

NegativeArraySizeException

```
public class Demo {
    public static void main(String[] args) {
        int a[] = new int[-5];
    }
}
```

Exception Handling Keywords

try

Contains code that may generate an exception

```
try {
    int result = 10 / 0;
}
```

catch

Handles the exception

```
try {
    int result = 10 / 0;
}
catch (ArithmeticException e) {
    System.out.println("Exception handled.");
}
```

finally

Always executes whether an exception occurs or not.

```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10 / 0;
        }
        catch (ArithmeticException e) {
            System.out.println("Error occurred");
        }
        finally {
            System.out.println("Finally block executed");
```

```
        }
    }
}
```
Error occurred
Finally block executed

Throw: Explicitly throws an exception

```
public class Demo {
    public static void main(String[] args) {
        int age = 15;
        if (age < 18) {
            throw new IllegalArgumentException("Age must be 18 or above");
        }
        System.out.println("Eligible");
    }
}
```
Exception in thread "main"
java.lang.IllegalArgumentException: Age must be 18 or above

```
public class Demo {
    public static void main(String[] args) {
        int age = 15;
        try {
            if (age < 18) {
                throw new IllegalArgumentException("Age must be 18 or above");
            }
            System.out.println("Eligible");
        }
        catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("Program continues");
    }
}
```
Age must be 18 or above
Program continues

Throws: Declares that this method may throw an exception

```
import java.io.FileNotFoundException;
public class Main{
    static void checkFile() throws FileNotFoundException {
        throw new FileNotFoundException("data.txt not found");
    }
    public static void main(String[] args) {
```

```
        try {
            checkFile();
        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

data.txt not found

```
import java.io.IOException;
public class Main {
    // throws: Declares that this method may throw IOException
    static void checkAge(int age) throws IOException {
        if (age < 18) {
            // throw: Explicitly throws an exception
            throw new IOException("Age must be 18 or above.");
        }
        System.out.println("Eligible to vote.");
    }
    public static void main(String[] args) {
        try {
            checkAge(15);
        }
        catch (IOException e) {
            System.out.println(e.getMessage());
        }
        finally {
            System.out.println("Finally block executed.");
        }
        System.out.println("Program Ends.");
    }
}
```

Age must be 18 or above.
Finally block executed.
Program Ends.

Multiple Catch Blocks

```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
```

```
            int result = a / b;    // ArithmeticException
            int arr[] = {1, 2, 3};
            System.out.println(arr[5]);   // ArrayIndexOutOfBoundsException
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index is invalid.");
        } catch (Exception e) {
            System.out.println("Some other exception occurred.");
        }
        System.out.println("Program continues...");
    }
}
```

Cannot divide by zero.
Program continues...

User-Defined Exception

A user-defined exception (also called a custom exception) is an exception created by the programmer to handle application-specific errors.

Example 1: Invalid Age

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}
public class Main {
    static void checkAge(int age) throws InvalidAgeException {
        if(age < 18)
            throw new InvalidAgeException("Not Eligible to Vote");
        else
            System.out.println("Eligible to Vote");
    }
    public static void main(String args[]) {
        try {
            checkAge(16);
        }
        catch(InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Not Eligible to Vote

Example 2: Insufficient Balance

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}
public class Bank {
    static void withdraw(int balance, int amount)
            throws InsufficientBalanceException {
        if(amount > balance)
            throw new InsufficientBalanceException("Insufficient Balance");
        else
            System.out.println("Amount Withdrawn");
    }
    public static void main(String args[]) {
        try {
            withdraw(5000,6000);
        }
        catch(InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Insufficient Balance

Example 3: Invalid Password

```
class InvalidPasswordException extends Exception {
    InvalidPasswordException(String msg) {
        super(msg);
    }
}
public class Login {
    public static void main(String args[]) {
        String password = "abc123";
        try {
            if(!password.equals("admin123"))
                throw new InvalidPasswordException("Wrong Password");
            System.out.println("Login Successful");
        }
        catch(InvalidPasswordException e) {
            System.out.println(e.getMessage());
        }
```

```
    }
}
```